

上海电力大學

SHANGHAI UNIVERSITY OF ELECTRIC POWER

2026 年全国大学生计算机系统能力大赛

操作系统设计赛（内核实现赛道）

SWTC 初赛作品文档

面向官方评测的 RISC-V64 / LoongArch64 双架构操作系统内核

参赛学校	上海电力大学
参赛队伍	sudo_win_the_cscc
作品名称	SWTC
指导教师	徐曼
队伍成员	余子豪、王溢敏
文档日期	2026 年 6 月 30 日

初赛提交版

1. 作品概况

1.1 作品基本信息

项目	内容
队伍名称	sudo_win_the_cscc
操作系统名称	SWTC
支持架构	RISC-V64、LoongArch64
主要语言	Rust、Makefile、少量汇编与 Shell
官方产物	kernel-rv、kernel-la
构建方式	仓库根目录执行 make all，离线完成双架构构建
测试方向	basic、BusyBox、Lua、libcbench、libctest、LTP

1.2 项目人员

参赛身份	姓名	学校与专业
指导教师	徐曼	上海电力大学
队长、队员	余子豪	上海电力大学，计算机科学与技术（卓越计划）
队员	王溢敏	上海电力大学，计算机科学与技术（卓越计划）

1.3 作品概述

SWTC 是面向 2026
操作系统设计赛官方评测环境开发的双架构操作系统内核。项目在同一仓库、同一提交入口下维护 RISC-V64 与 LoongArch64 两条实现主线，目标是稳定生成官方要求的两个内核 ELF，并执行测试盘中的 Linux 用户态程序。

项目围绕官方无分区 EXT4 测试盘、预编译 ELF 和测试脚本构建端到端运行链路：内核启动后识别块设备，读取脚本与资源，组织测试队列，装载用户程序，提供最小 Linux ABI，按官方格式输出结果并主动关机。历史最高可见官网成绩为 983.6805892892955。

> 项目以“可构建、可启动、可执行、可计分、可复现”为初赛工程目标。

2. 系统总体架构

SWTC 采用统一提交入口与双架构实现并行的工程组织方式。根构建负责选择各自工具链、恢复离线依赖配置、检查源树完整性并生成官方产物；运行时则统一面向官方测试盘和日志协议。

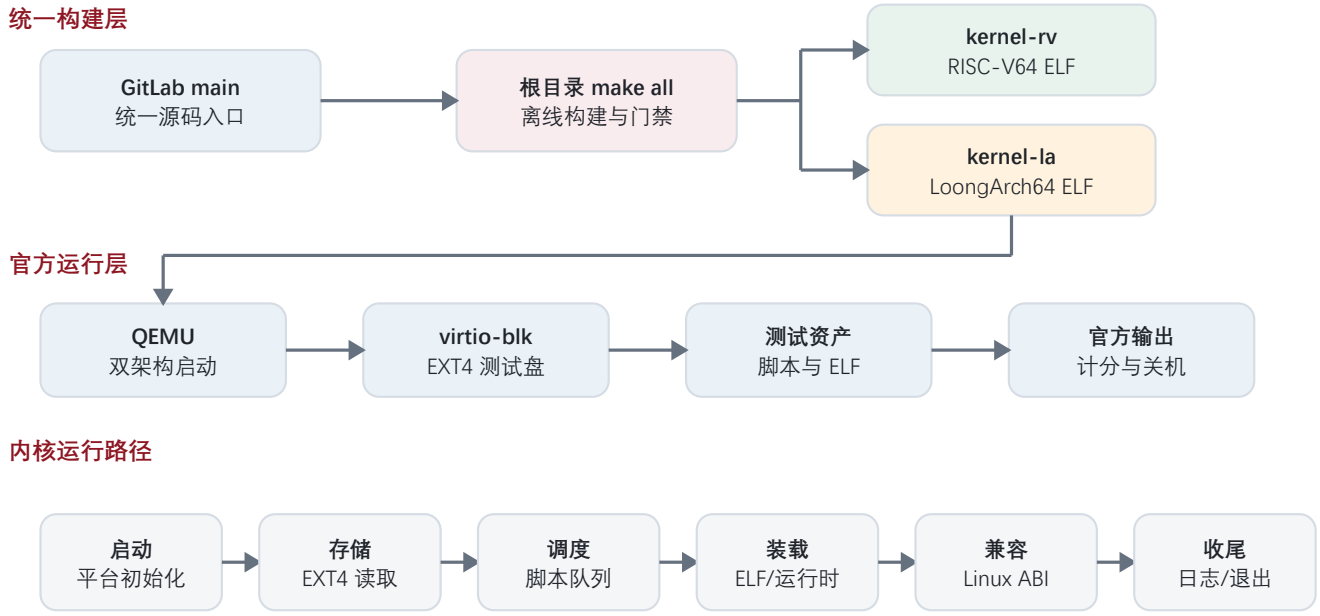


图 1 SWTC 统一构建、官方运行环境与内核运行路径

2.1 双架构组织

- RISC-V64 主线位于 SWTC/，以 Titanix 往届开源架构为基础，重点完成官方镜像入口、Linux ABI、EXT4 读取、脚本解析和测试执行适配。
- LoongArch64 主线位于 SWTC-la/，基于 StarryX/ArceOS 组件化内核进行比赛适配，重点完成工具链、真实 kernel-la、测试盘和用户态应用运行。
- 两条主线共享官方提交约束、测试组命名、日志格式、回归标准与文档记录方式。

2.2 核心数据流

阶段	输入	核心处理	输出
构建	GitLab 源码	双工具链离线构建、源树门禁	kernel-rv / kernel-la
发现	virtio-blk	识别无分区 EXT4 与固定脚本路径	测试组与资源索引
执行	脚本、ELF、运行时	staging、ELF 装载、进程与 syscall	测试结果
收尾	case 状态	官方 START/END 格式、超时隔离	计分日志与主动关机

3. 主要工作内容

3.1 官方构建入口与离线依赖

- 根目录保留 Makefile 与 GNUmakefile，执行 make all 统一生成两个官方 ELF。
- RISC-V 与 LoongArch 使用各自固定 nightly 工具链，避免跨架构工具链和 target 配置相互污染。
- 针对官方提交过滤隐藏文件的约束，构建时从非隐藏目录恢复 Cargo 配置与 checksum，并对 vendor 完整性进行检查。
- LoongArch 构建默认拒绝使用 RISC-V 产物充当 kernel-la，使架构错误在编译阶段直接暴露。

3.2 官方测试盘与脚本执行

- 通过 virtio-blk 访问无分区 EXT4 测试盘，支持多级目录下普通文件与脚本的只读访问。
- 按 glibc、musl 和根目录路径定位测试组，解析脚本并形成可控的串行执行队列。
- 将 ELF、动态解释器、脚本和数据文件暂存到组私有目录，降低跨组资源污染。
- 对高风险测试采用逐 case 超时和组级 START/END 收尾，防止单项卡死导致整轮评测失效。

3.3 Linux ABI 与进程执行

- 支持 ELF PT_LOAD/PT_INTERP 处理，构造 argc、argv、envp 与 auxv，并执行动态或静态用户程序。
- 逐步实现文件描述符、路径、内存、进程、时间、信号与 futex 等 Linux syscall 兼容语义。
- 建立 fork/clone、execve、wait4 与超时控制的最小执行链，满足测试程序父子进程和线程场景。
- 根据线上日志和本地镜像结果定位缺失 ABI，按可验证的小批次扩展 LTP 覆盖。

3.4 LoongArch64 实现与适配

- 完成 LoongArch64 内核启动、离线 vendor、配置生成器与 GCC 13.2 musl 工具链适配。
- 真实 kernel-la 为 LoongArch ELF，入口地址 0x80000000；本地已验证 basic 与 BusyBox 测试组可运行并主动退出。
- 将易受官方源包过滤影响的核心依赖迁移到非隐藏目录，并在根构建中增加源树完整性门禁。
- 对测试顺序与工作目录进行隔离，避免 BusyBox 递归扫描大型 LTP 目录而中断后续测试。

4. 完成度与验证状态

以下状态以已完成的本地回归和可见官网结果为依据。线上得分受评测源包、排队与运行环境影响，因此文档将“实现完成”“本地验证”和“线上计分”分开描述。

测试/模块	RISC-V64	LoongArch64	当前说明
根构建	已通过	已通过	干净导出可生成两个架构正确的 ELF
basic	线上稳定得分	本地 64/64，曾线上得分	双 libc 路径已接入
BusyBox	线上得分	本地 55/55，曾线上得分	LA 已隔离工作目录
Lua	线上得分	已接入	依赖 Lua 与脚本资源 staging
libcbench	线上得分	部分接入	高风险 glibc 项已后置隔离
libctest	musl 线上得分	已接入	LA 逐 case 继续验证
LTP	稳定子集线上得分	部分接入	按 syscall 能力分批扩展
EXT4/tmpfs	已接入	已接入	只读测试盘 + 运行资源暂存
进程模型	最小 Linux ABI	组件化内核适配	尚非完整 Linux 隔离模型

4.1 已建立的回归门禁

- 干净导出构建：确保源码包不依赖本机未提交文件或缓存。
- ELF 架构检查：kernel-rv 必须为 RISC-V ELF，kernel-la 必须为 LoongArch ELF。
- 无盘 QEMU 回归：没有测试盘时也能启动、输出并主动退出。
- 官方镜像回归：检查测试组 START/END、case 输出、超时行为与最终关机。
- 线上结果回读：以官网分项和串口日志更新进度，避免凭本地现象推断线上结论。

历史最高可见官网成绩：983.6805892892955（2026-06-29）。

5. 项目特点与创新点

方向	实现与价值
双架构统一提交	在同一仓库维护 RISC-V64 与 LoongArch64 两套内核、依赖和工具链，根构建统一产出官方要求的两个 ELF。
测试脚本受控执行	不依赖完整发行版环境，将官方脚本解析为内核可调度的命令队列，统一管理资源、顺序、超时和日志。
评测约束工程化	围绕隐藏文件过滤、离线构建、源树缺失和长排队等现实约束，设计非隐藏依赖、完整性门禁和干净导出验证。
日志驱动 ABI 演进	根据真实 ELF 行为和线上日志逐项补齐 Linux syscall 语义，以可回归的小批次扩展 functional 测试覆盖。
真实架构产物防线	禁止用 RISC-V ELF 冒充 kernel-la，构建阶段检查源树与 ELF 架构，使错误尽早暴露。
测试组故障隔离	通过组私有工作目录、逐 case 超时和高风险项后置，减少单个测试对整轮成绩的级联影响。

5.1 工程取舍

初赛阶段优先保证官方入口、Linux ABI 和 functional 测试链路，不将网络、图形界面等展示性功能置于计分闭环之前。对于尚未稳定的 iozone、lmbench 和大规模 LTP，采用后置、分批和可回滚策略，避免一次改动破坏已验证基线。

> 创新重点不在堆叠功能数量，而在双架构、真实测试资产和受限评测环境下建立稳定、可解释、可复现的内核运行链路。

6. 官网评测记录

下图为 2026 年 6 月 30 日官网评测列表截图。红框行显示参赛队伍 sudo_win_the_cscc，学校为上海电力大学；多个分项列已获得稳定结果，证明 SWTC 已能进入官方评测链路并执行双架构相关测试。

63	T2026104869910402	StellaOS	武汉大学	69	2026-06-30 12:00:15	102.0	102.0	102.0	102.0	54.0	54.0	53.0	53.0
64	T202610055999994	AYGNEP	南开大学	47	2026-06-23 15:23:33	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
65	T2026101269910482	一起造火箭	内蒙古大学	17	2026-06-30 11:04:58	102.0	102.0	102.0	102.0	55.0	55.0	55.0	55.0
66	T202610486999797	DWG	武汉大学	71	2026-06-30 15:01:12	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
67	T2026101269910295	再来两次	内蒙古大学	115	2026-06-30 10:55:47	102.0	102.0	102.0	102.0	53.0	54.0	53.0	54.0
68	T2026101269910207	歪比巴卜	内蒙古大学	134	2026-06-30 14:51:36	100.0	102.0	100.0	102.0	54.0	54.0	54.0	54.0
69	T2026102569910192	sudo_win_the_cscc	上海电力大学	122	2026-06-30 09:17:33	102.0	102.0	102.0	102.0	54.0	50.0	54.0	50.0
70	T2026101839910132	不想写注释队	吉林大学	99	2026-06-27 15:10:37	102.0	95.0	102.0	95.0	51.0	50.0	54.0	53.0
71	T202610487999858	httos	华中科技大学	26	2026-06-29 17:57:54	102.0	102.0	102.0	102.0	49.0	52.0	49.0	52.0
72	T2026102699910462	0xDEADBEEF	华东师范大学	182	2026-06-28 23:14:09	97.0	97.0	97.0	97.0	54.0	54.0	54.0	54.0
73	T2026100079911061	pilot.00	北京理工大学	17	2026-06-30 14:01:51	0.0	0.0	0.0	102.0	0.0	0.0	0.0	54.0
74	T2026106109910526	XCX	四川大学	33	2026-06-25 18:14:25	0.0	102.0	0.0	98.0	0.0	54.0	0.0	54.0
75	T202610701999762	NameNotFound	西安电子科技大学	26	2026-05-04 16:56:32	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0

图 2 sudo_win_the_cscc 官网评测列表记录（2026-06-30）

评测成绩是阶段性工程证据，不等同于完整 Linux 兼容度。项目仍通过本地官方镜像回归和线上分项日志交叉验证每项能力。

7. 构建、验证与后续计划

7.1 构建方式

```
cd /mnt/e/WorkSpace/MinOS/os-kernel-contest
make all
```

- kernel-rv：RISC-V executable ELF，入口地址 0x80200000。
- kernel-la：LoongArch executable ELF，入口地址 0x80000000。
- 构建默认使用仓库内离线依赖，不要求评测机访问 crates.io。

7.2 当前边界

边界	后续处理
进程与资源隔离仍是最小实现	继续完善 per-process fd、父子资源语义和独立内核栈
LoongArch libctest/LTP 覆盖不足	按 case 日志补齐 syscall 与运行时语义
高风险性能测试可能截断后续组	保持后置、限时和独立工作目录
双架构实现基础不同	统一测试协议、构建门禁和文档，逐步收敛公共行为

7.3 后续计划

优先级	工作项	验收标准
P0	稳定双架构官方构建与启动	两个 ELF 架构正确，QEMU 可进入内核并主动退出
P1	恢复并巩固已验证高分路径	basic/BusyBox/Lua/libcbench 分项不回退
P2	扩展 LA libctest 与 LTP	逐 case 可解释，新增 syscall 有本地和线上证据
P3	完善进程与文件描述符模型	fork/exec/wait 与多程序资源隔离回归通过

8. 参考、开源基础与致谢

SWTC 在遵守原项目许可证的前提下学习并复用开源操作系统的架构基础。RISC-V 主线以 Titanix 往届 GPLv3 开源项目为基础；LoongArch 主线参考 StarryX 与 ArceOS 组件化设计。本队增量主要集中在 2026 官方评测适配、双架构构建、Linux ABI 修复、测试调度和离线构建工程化。

[1] Titanix：https://github.com/greenhandzpx/Titanix

[2] StarryX：https://github.com/Anekoique/StarryX

[3] NoAxiom-OS：https://github.com/NoAxiom/NoAxiom-OS

[4] NPUcore-IMPACT：https://github.com/Fediory/NPUcore-IMPACT

[5] UMI：https://github.com/js2xxx/umi

[6] Alien：https://github.com/Godones/Alien

[7] NPUcore：https://github.com/echodjx/NPUcore

[8] UltraOS：https://github.com/xiyurain/UltraOS

[9] Del0n1x / myOS：https://github.com/Ya0rk/myOS

[10] Chronix：https://github.com/PACTHEMAN123/Chronix

开源与原创性说明

项目保留所使用开源组件的许可证和来源说明，不直接复制未授权实现。文档所述比赛适配、测试组织、构建门禁和 ABI 修复以本仓库提交记录与验证材料为依据。

— 文档结束 —